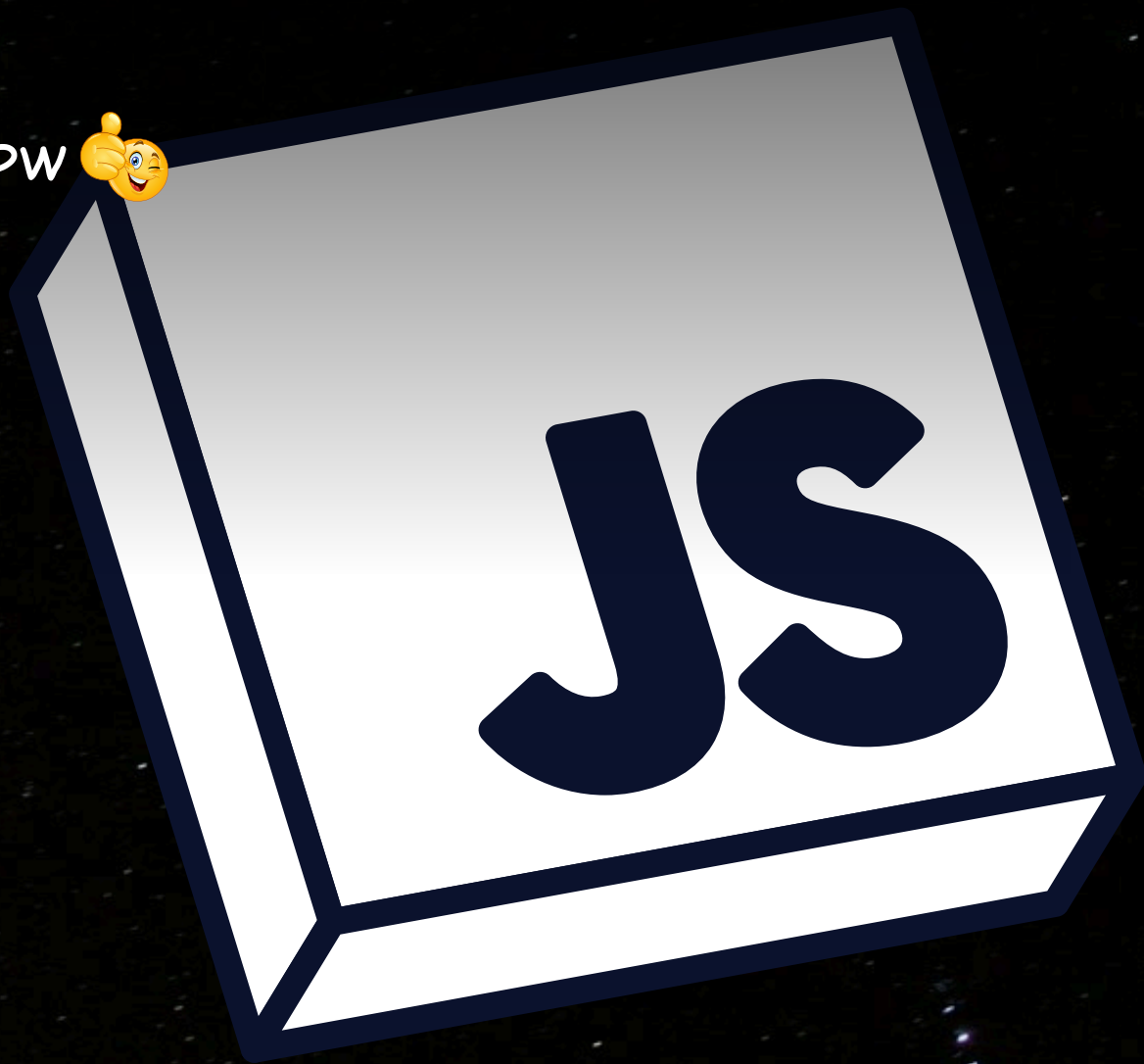


FREELANCER

Javascript functions that are often underrated

Save it now 🤔



@shadowctrl
Freelance Developer

www.shadowctrl.me

FREELANCER

Debounce()

Your Best Friend for Search Inputs

```
// Your code goes here
const debounce = (func, wait) => {
  let timeout;
  return (...args) => {
    clearTimeout(timeout);
    timeout = setTimeout(() => func(...args), wait);
  };
}

const handleSearch = debounce((searchTerm) => {
  // API call or heavy computation
}, 300);
```

Delays function execution for improved search responsiveness.



@shadowctrl
Freelance Developer

www.shadowctrl.me

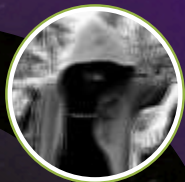
FREELANCER

memoize()

Cache Expensive Operations

```
// Your code goes here
const memoize = (fn) => {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}
```

caches function outputs, avoiding
redundant computations.



@shadowctrl
Freelance Developer

www.shadowctrl.me

FREELANCER

deepclone()

Safe Object Copying

// Your code goes here

```
const deepClone = (obj) => {  
  if (obj === null || typeof obj !== 'object') return obj;  
  const copy = Array.isArray(obj) ? [] : {};  
  Object.keys(obj).forEach(key => {  
    copy[key] = deepClone(obj[key]);  
  });  
  return copy;  
}
```

safely copies complex objects, avoiding references.



@shadowctrl
Freelance Developer

www.shadowctrl.me

FREELANCER

retry()

Resilient API Calls

```
// Your code goes here

const retry = async (fn, retries = 3, delay = 1000) => {
  try {
    return await fn();
  } catch (error) {
    if (retries <= 0) throw error;
    await new Promise(resolve => setTimeout(resolve, delay));
    return retry(fn, retries - 1, delay);
  }
}
```

Executes API calls reliably, handling failures.



@shadowctrl
Freelance Developer

www.shadowctrl.me

FREELANCER

groupBy()

Data Organization Made Simple

```
// Your code goes here
```

```
const groupBy = (array, key) => {  
  return array.reduce((result, item) => {  
    (result[item[key]] = result[item[key]] || []).push(item);  
    return result;  
  }, {});  
}
```

A powerful tool for organizing data by grouping elements with common properties.



@shadowctrl
Freelance Developer

www.shadowctrl.me

Did you find it Useful?

Leave a **comment!**



Alamin

@CodeWithAlamin

FOLLOW FOR MORE



Like



Comment



Repost